FIRESIDE CHAT · PREPARED FOR THE CHIEF DATA OFFICE · LEAD DATA ENGINEER

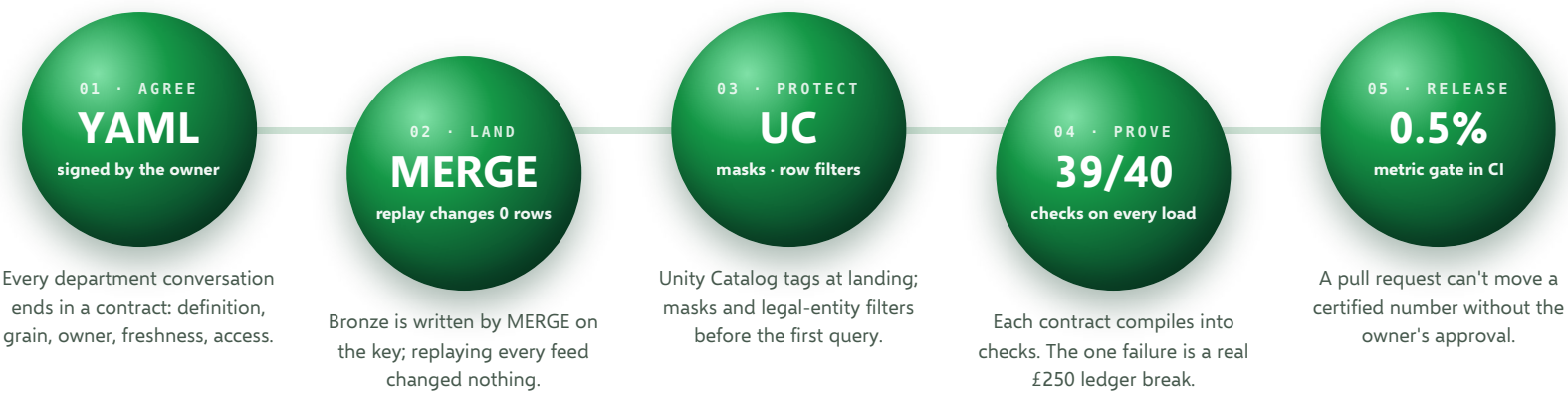
Nexus, **for Quilter.**

One contract per number. Governed by default. Tested before trust.

NEXUS is the platform I designed and ran at Lifepal: Databricks within Azure, contract-driven and tested end to end, turning raw operational data into numbers the business could act on without asking twice. Here it is applied to Quilter, so every department's number is agreed once with its owner, landed replay-safe, protected in Unity Catalog, and checked on every load. Everything below runs at **rfim.github.io/nexus-quilter**.



BUILT AT lifepal → PRESENTED TO Quilter



Every department conversation ends in a contract: definition, grain, owner, freshness, access.

Bronze is written by MERGE on the key; replaying every feed changed nothing.

Unity Catalog tags at landing; masks and legal-entity filters before the first query.

Each contract compiles into checks. The one failure is a real £250 ledger break.

A pull request can't move a certified number without the owner's approval.

PROVEN AT LIFEPAL · **6h → 90m** batch · **24h → <5m** latency · **512** assertions, 47 sources · **−60%** ad-hoc requests · **$11k** vs $19k budget

IF I JOINED YOUR TEAM

the first 90 days

1 DAYS 1–30

Listen and inventory

Learn the domain from its owners and its data. Build a queryable record of what exists, what it feeds and who owns it. Change nothing.

2 DAYS 31–60

Own the busiest products

Run them, fix them and join their incident rota. Write contracts for the rules that live in someone's head.

3 DAYS 61–90

Standardise

One ingestion pattern, one quality mechanism, one deployment path, agreed with the other leads and adopted where each has proved itself twice.

4 DAY 90

One product, end to end

A certified product with its contract, SLO, runbook and metric gate, in front of its owner and in use.

> **Every day after that:** I lead the team and the platform (design it, ship it, grow the team) and serve the business (the departments, on-call, Consumer Duty, real time, self-service). Each one is a chapter you can open at rfim.github.io/nexus-quilter.

DATABRICKS, FEATURE BY FEATURE

BUILT AT NEXUS · LIFEPAL	DATABRICKS WITHIN AZURE, IN PRODUCTION
Delta MERGE	Every feed lands by MERGE on its key, newest CDC sequence wins. Replaying every feed changed **0 rows**.
SCD2 + as-of	Fund charges kept as history and joined at the trade date, never today's price.
expectations	The source contract becomes Lakeflow expectations; bad rows go to quarantine with the rule attached: **15 of 13,022**.
Unity Catalog	Tags at landing, column masks, legal-entity row filters, grants generated from each contract.
Asset Bundles	One job definition for dev and prod, deployed by CI. Nobody edits a job in the UI.
metric gate	A pull request fails if a certified number moves **more than 0.5%**.
metric views	A definition enters the metric view first, the report second: **ad-hoc requests −60%**.
agents	AI reads governed functions with the caller's own permissions, never a service account.

`data contract`

The department owns the definition; the platform checks it. This YAML compiles into SQL checks, Lakeflow expectations and Unity Catalog grants.

```yaml
product:   finance_net_flows_daily
owner:     finance-reporting-lead@example.com  # signs off
grain:     [legal_entity, wrapper_code, trade_date]
freshness: {column: trade_date, max_lag_days: 1}  # by 06:30
reconcile:
  against:       finance_ledger
  match_on:      [trade_date, wrapper_code]
  tolerance_gbp: 0.01
access:
  read:    [grp_finance, grp_exec_mi]
change:
  breaking: owner_approval   # blast radius posted on the PR
```

`CI gate`

Every pull request runs the tests, validates the bundle and diffs certified numbers against production.

```yaml
on: pull_request
steps:
  - run: pytest -q          # replay, SCD2, contracts
  - run: databricks bundle validate -t prod
  - run: python -m nexus.gate --max-diff 0.5  # certified
```

NUMBERS, DONE HONESTLY

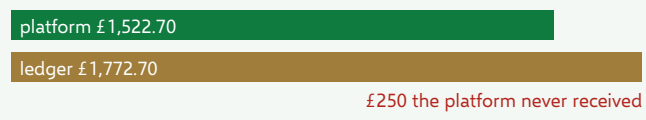
worked examples from the run

Reconciliation

£250 of £9.44m

$$|\ \Sigma \text{ platform} - \text{ledger}\ | \le £0.01 \ \rightarrow\ \textbf{1 day fails}$$

14 Aug 2026 · ISA



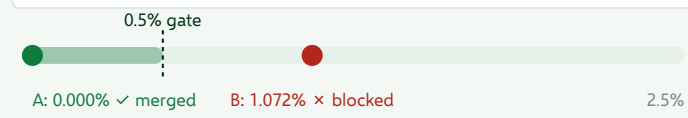
£250 the platform never received

One posting the ledger has and the platform never sent. 0.003% of net flows, so the board pack goes ahead with a note, and the check stays red until the source resends.

Metric gate

blocked at 1.07%

$$\Delta = |\text{cand} - \text{cert}| \div \text{cert} = \textbf{1.072\%} > 0.5\%$$

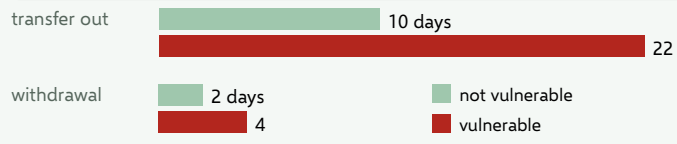


Version B of a pull request forgot that fees count towards net flows. The gate computed the diff on the Delta tables and blocked the merge.

Consumer Duty

gap 2.2×

$$\tilde{x}_{\text{vulnerable}} \div \tilde{x}_{\text{others}} = 22 \div 10 = \textbf{2.2×} > 1.25$$



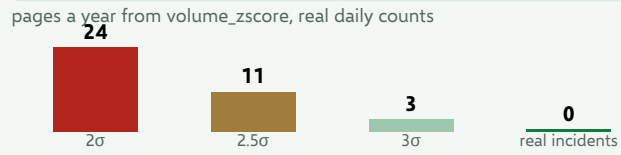
Published as an aggregate with a minimum cell size of 10 (smallest group: 29), so the analyst sees the gap without seeing anyone's flag.

Alerting

page on SLOs

$$z = (n - \mu_{28d}) \div \sigma_{28d}$$

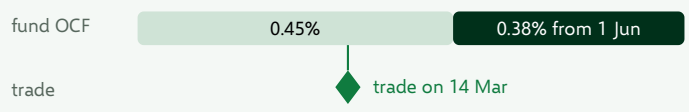
pages a year from volume_zscore, real daily counts



A volume alert at 2σ would page 24 times a year with nothing behind it. Volume goes to a morning digest; pages are for SLO breaches with a runbook.

As-of joins

33 rows restated



✓ as-of join: 0.45%, the charge in force on the day
× current join: 0.38%, a price change the client never paid

Facts join the fund charge in force on the trade date. When 40 late corrections arrived, 33 Finance rows changed, and `VERSION AS OF 0` still shows what was issued.

THE CODE BEHIND IT

github.com/rfim/nexus-quilter

`Delta MERGE · src/nexus/bronze.py`

Bronze is written by MERGE, never INSERT, so a replayed batch, or a job re-run after a failure, **changes nothing**.

```python
def upsert_bronze(spark, batch, cfg):
    keys, order, target = cfg["merge_keys"], cfg["ordering_key"], cfg["table"]
    # dedupe inside the batch, so a replayed offset range can't apply twice
    w = Window.partitionBy(*keys).orderBy(F.col(order).desc())
    latest = batch.withColumn("_rn", F.row_number().over(w)).filter("_rn = 1")
    cond = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (DeltaTable.forName(spark, target).alias("t")
        .merge(latest.alias("s"), cond)
        .whenMatchedUpdateAll(condition=f"s.{order} > t.{order}")  # newest wins
        .whenNotMatchedInsertAll()
        .execute())
```

`Unity Catalog · generated SQL`

Protection comes from the contracts and lives with the data, so BI, notebooks and agents all get the same rules.

```sql
-- generated from the contracts: protection lives with the data
ALTER TABLE nexus.silver.dim_client
  ALTER COLUMN email SET MASK nexus.gov.mask_pii;
CREATE FUNCTION nexus.gov.entity_filter(legal_entity STRING)
  RETURN is_account_group_member('grp_exec_mi_all')
      OR is_account_group_member(concat('grp_', legal_entity));
ALTER TABLE nexus.gold.finance_net_flows_daily
  SET ROW FILTER nexus.gov.entity_filter ON (legal_entity);
GRANT SELECT ON TABLE nexus.gold.finance_net_flows_daily
  TO grp_finance;
```

> **Stated plainly.** Lifepal outcomes come from my CV and production work. The Quilter side is built from outside, from the role and public information, on synthetic data; no Quilter systems or data were used. The numbers in the worked examples are from that synthetic run.

R. FIRMAN INSAN MUHAMMAD (VIM) · LEAD DATA ENGINEER · LONDON · FIRMAN.INSAN.M@GMAIL.COM RFIM.GITHUB.IO/NEXUS-QUILTER · 14 OCTOBER 2026